



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验设计报告

开课学期: 2024 年秋季
课程名称: 操作系统
实验名称: 进程与系统调用
实验性质: 课内实验
实验时间: 10.18 地点: T2608
学生班级: 5
学生学号: 220110515
学生姓名: 金正达
评阅教师: _____
报告成绩: _____

实验与创新实践教育中心印制

2024 年 9 月

一、 回答问题

1. 阅读 `kernel/syscall.c`, 试解释函数 `syscall()` 如何根据系统调用号调用对应的系统调用处理函数（例如 `sys_fork`）？`syscall()` 将具体系统调用的返回值存放在哪里？

`syscall()` 函数根据系统调用号在存入好所有索引-函数地址的 `syscalls` 数组中调用对应的系统调用处理函数。系统调用的返回值被 `syscall()` 函数放在了调用进程的 `a0` 寄存器中。

2. 阅读 `kernel/syscall.c`, 哪些函数用于传递系统调用参数？试解释 `argraw()` 函数的含义。

`argraw`、`argint`、`argaddr`、`argstr` 被用于传递系统调用参数。
`argraw()` 函数用于根据传递参数的索引选择返回对应的寄存器值。

3. 阅读 `kernel/proc.c` 和 `proc.h`, 进程控制块存储在哪个数组中？进程控制块中哪个成员指示了进程的状态？一共有哪些状态？

进程控制块储存在 `proc` 结构体之中。`proc` 中的 `state` 变量指示了当前进程的状态，状态包括：UNUSED、SLEEPING、RUNNABLE、RUNNING、ZOMBIE。

4. 在任务一当中，为什么子进程（4、5、6 号进程）的输出之前会 **稳定的** 出现一个 `$` 符号？（提示：`shell` 程序(`sh.c`)中什么时候打印出 `$` 符号？）

```
$ exittest
exit test
[INFO] proc 3 exit, parent pid 2, name sh, state sleep
[INFO] proc 3 exit, child 0, pid 4, name child0, state sleep
[INFO] proc 3 exit, child 1, pid 5, name child1, state sleep
[INFO] proc 3 exit, child 2, pid 6, name child2, state sleep
$ [INFO] proc 4 exit, parent pid 1, name init, state runble
[INFO] proc 5 exit, parent pid 1, name init, state runble
[INFO] proc 6 exit, parent pid 1, name init, state runble
```

进程 3 退出后，控制权交给进程 3 的父进程 `shell`，`shell` 进程等待下一个命令执行，所以会打印一个 `$`。

5. 在任务三当中，我们提到测试时需要指定 CPU 的数量为 1，因为如果 CPU 数量大于 1 的话，输出结果会出现乱码，这是为什么呢？（提示：多核心调度和单核心调度有什么区别？）

在多核心调度时，同一时刻会有多个进程运行，多个进程可能会同时访问输出流，导致打印内容交错。单核心调度时，同一时刻只有一个进程在运行，输出流也只有一个进程访问。

二、实验详细设计

注意不要照搬实验指导书上的内容，请根据你自己的设计方案来填写

1. exit

在 exit 函数中打印当前进程信息，在 reparent 函数中打印当前进程的子进程的信息。

2. wait

在 sys_wait 函数中使用 argaddr 获取用户程序传递的 flags 参数，从而调用 wait

```

21  uint64 sys_wait(void) {
22      uint64 p;
23      uint64 flags;
24      argaddr(1, &flags);
25      if (argaddr(0, &p) < 0) return -1;
26      if (flags) return wait(p, 1);
27      return wait(p, 0);
28  }

```

wait 中增加一个判断，当 flags 为 1 时直接返回-1，在返回前解锁。

3. yield

增加 sys_yield 函数，其内容为调用 yield 函数；

在 usys.pl 中增加函数 yield 调用入口；

在 syscall.h 中增加系统调用号；

在 syscall.c 中增加调用定义，将系统调用号与函数绑定；

在 sys_yield 函数中增加当前进程上下文保存信息和当前进程信息，并且寻找下面的状态为 RUNNABLE 的进程作为即将运行进程，打印它们的信息。

```

// sys_yield, only call yield in proc.c
uint64 sys_yield(void) {
    struct proc* p = myproc();
    printf("Save the context of the process to the memory region from address %p to %p\n",
           p->context.sp, p->context.sp + sizeof(struct context));
    printf("Current running process pid is %d and user pc is %p\n", p->pid, p->trapframe->epc);
    // get the next proc which will be running
    for (struct proc *c = proc; c < &proc[NPROC]; c++) {
        acquire(&c->lock);
        if (c->state == RUNNABLE) {
            printf("Next runnable process pid is %d and user pc is %p\n", c->pid, c->trapframe->epc);
        }
        release(&c->lock);
    }
    yield();
    return 0;
}

```

三、 实验结果截图

exit:

```
xv6 kernel is booting

init: starting sh
$ exittest
exit test
proc 3 exit, parent pid 2, name sh, state running
proc 3 exit, child 0, pid 4, name child0, state runnable
proc 3 exit, child 1, pid 5, name child1, state runnable
proc 3 exit, child 2, pid 6, name child2, state runnable
$ proc 4 exit, parent pid 1, name init, state running
proc 5 exit, parent pid 1, name init, state running
proc 6 exit, parent pid 1, name init, state running
```

wait:

```
init: starting sh
$ waittest
wait test
no child exited yet, round 0
no child exited yet, round 1
no child exited yet, round 2
proc 4 exit, parent pid 3, name waittest, state running
child exited, pid 4
wait test OK
proc 3 exit, parent pid 2, name sh, state running
```

yield:

```
$ yieldtest
yield test
parent yield
Save the context of the process to the memory region from address 0x000003ffff9a70 to 0x000003ffff9ae0
Current running process pid is 5 and user pc is 0x00000000000003e2
Next runnable process pid is 6 and user pc is 0x0000000000000332
Next runnable process pid is 7 and user pc is 0x0000000000000332
Next runnable process pid is 8 and user pc is 0x0000000000000332
Child with PID 6 begins to run
proc 6 exit, parent pid 5, name yieldtest, state running
Child with PID 7 begins to run
proc 7 exit, parent pid 5, name yieldtest, state running
Child with PID 8 begins to run
proc 8 exit, parent pid 5, name yieldtest, state running
parent yield finished
proc 5 exit, parent pid 2, name sh, state running
```

make grade:

```
$ make qemu-gdb LAB_SYSCALL_TEST=on
exit test: OK (4.5s)
== Test wait test ==
$ make qemu-gdb LAB_SYSCALL_TEST=on
wait test: OK (1.8s)
== Test yield test ==
$ make qemu-gdb CPUS=1 LAB_SYSCALL_TEST=on
yield test: OK (0.7s)
Score: 100/100
```